

Intégration numérique d'une équation différentielle

4 septembre 2026

L'objectif de ce chapitre est d'étudier les méthodes d'intégration numérique d'une équation différentielle linéaire dépendante du temps et d'ordre n , d'en comprendre l'implémentation dans un langage tel que Matlab, ainsi que d'en analyser la stabilité et la précision.

1 Equation différentielle d'ordre n

Une équation différentielle est une relation algébrique qui lie la fonction inconnue $u(t)$ à ses dérivées, pouvant dépendre du temps t .

$$g(u, \frac{du}{dt}, \frac{d^2u}{dt^2}, \dots, \frac{d^nu}{dt^n}, t) = 0 \quad (1)$$

Son ordre n correspond au degré maximal de dérivation auquel la fonction inconnue est soumise. La relation (1) peut être linéaire comme dans le cas de l'oscillateur harmonique amorti :

$$\frac{d^2x}{dt^2} + \eta \frac{dx}{dt} + \omega_0^2 x = 0 \quad (2)$$

ou non-linéaire comme le cas d'un pendule pesant :

$$\frac{d^2\Theta}{dt^2} + \frac{g}{l} \sin \Theta = 0 \quad (3)$$

2 Equivalence entre Equation différentielle et système

Comme il n'existe pas de méthode générale pour résoudre une équation différentielle d'ordre $n > 1$, on essaie d'abord de la transformer en un système de n équations du premier ordre.

On introduit ainsi, des variables intermédiaires, dérivées successives de l'inconnue $u(t)$:

$$\begin{cases} Y^{(0)} & = u(t) \\ Y^{(1)} & = \frac{dY^{(0)}}{dt} = \frac{du(t)}{dt} \\ \dots & = \dots \\ Y^{(n-1)} & = \frac{dY^{(n-2)}}{dt} = \frac{d^{n-1}u(t)}{dt^{n-1}} \\ g(Y^{(0)}, Y^{(1)}, \dots, Y^{(n-1)}, t) & = 0 \end{cases} \quad (4)$$

La dernière ligne de (6) n'est que la réécriture de la relation (1) où les dérivées de l'inconnue $u(t)$ ont été remplacées par leur variable intermédiaire. Le problème est bien posé si le vecteur $Y(t) = (Y^0(t), Y^1(t), \dots, Y^{n-1}(t))$ est connu à l'instant initial.

De plus, lors de l'intégration numérique, il faudra tenir compte des contraintes imposées par le domaine de définition dans lequel évolue le vecteur $Y(t)$. Dans la suite, on supposera que le système peut s'écrire selon une forme explicite :

$$[J(Y(t))]\frac{dY}{dt} = F(t, Y(t)) \text{ ou encore } \frac{dY}{dt} = f(t, Y(t)) \quad (5)$$

en supposant la matrice $[J]$ inversible à tout instant.

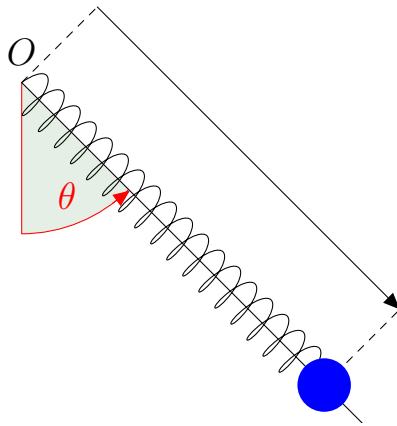
2.1 Application à l'oscillateur harmonique

On transforme d'abord l'équation différentielle (2) qui est d'ordre 2 en un système de 2 équations différentielles d'ordre 1, en introduisant la variable intermédiaire vitesse.

$$\begin{cases} \frac{dx}{dt} = v \\ \frac{dv}{dt} = -\omega_0^2 x - \eta v \end{cases} \text{ ou encore } \frac{d}{dt} \begin{pmatrix} x \\ v \end{pmatrix} = \begin{pmatrix} v \\ -\omega_0^2 x - \eta v \end{pmatrix} = f(Y(t)) \quad (6)$$

Le vecteur inconnu $Y(t)$ est alors formé de $(x(t), v(t))^t$ dont les valeurs doivent être données à l'instant initial.

2.2 Application au pendule élastique pesant



Le système est formé d'un anneau de masse m relié à un ressort, pouvant coulisser le long d'une barre de masse négligeable. On désire étudier le mouvement d'oscillation de ce système mixte pendule-ressort autour d'un axe O .

En notant r et θ , les positions radiale et angulaire de l'anneau, on montre que le mouvement du pendule élastique pesant est régi par un système de deux équations différentielles d'ordre 2 :

$$\begin{cases} \ddot{r} - r\dot{\theta}^2 &= -\frac{k}{m}(r - l_0) + g \cos \theta \\ r\ddot{\theta} + 2\dot{r}\dot{\theta} &= -g \sin \theta \end{cases} \quad (7)$$

FIGURE 1 – système mécanique

où l_0 est la longueur à vide du ressort. Pour le transformer en un système d'ordre 1, on introduit la vitesse radiale $v = \dot{r}$ et la vitesse angulaire $\omega = \dot{\theta}$. On obtient aisément :

$$\begin{cases} \frac{dr}{dt} &= v \\ \frac{dv}{dt} &= r\omega^2 - \frac{k}{m}(r - l_0) + g \cos \theta \\ \frac{d\theta}{dt} &= \omega \\ \frac{d\omega}{dt} &= -2 v \omega / r - g \sin \theta / r \quad \text{ssi } r > 0 \end{cases} \quad (8)$$

Le vecteur inconnu s'écrit $Y(t) = (r, v, \theta, \omega)^t$ et $f(Y(t))$ correspond au second membre de l'équation (8). La dernière ligne laisse apparaître la condition $r > 0$ qu'il faudra obligatoirement respecter lors de l'intégration de ce système.

3 Intégrateurs géométriques explicites à pas séparés

On discrétise d'abord le temps physique t en petits pas de temps constants k tels que $t_n = nk$. Le principe de l'intégration numérique est de construire la fonction $Y(t)$ point par point. Connaissant la valeur de Y_n , on va calculer la valeur Y_{n+1} à l'instant $n + 1$. Si nécessaire, on introduit des instants intermédiaires afin de pouvoir approcher les dérivées qui interviennent dans les développements de Taylor afin d'obtenir la précision souhaitée.

3.1 Euler (Runge-Kutta d'ordre 1)

Le développement de Taylor de Y_{n+1} au voisinage de l'instant n s'écrit :

$$Y_{n+1} = Y_n + k \left. \frac{dY}{dt} \right|_n + \vartheta(k^2) \quad (9)$$

La valeur de l'inconnue est obtenue à chaque pas d'intégration $n > 0$ en appliquant la relation de récurrence :

$$Y_{n+1} = Y_n + k f(t_n, Y_n) \quad (10)$$

à condition de connaître Y_0 .

On note que l'erreur locale à chaque pas, due à la troncature de la série, est donnée par $\epsilon_n = \frac{k^2}{2} \left. \frac{d^2 Y}{dt^2} \right|_n$. Au bout d'un temps T , correspondant à T/k pas d'intégration, l'erreur globale du schéma est donc en $T/k \vartheta(k^2) \propto \vartheta(k)$. On en déduit que l'ordre de la méthode d'Euler est 1.

3.1.1 Exemple d'application

On considère l'équation non-linéaire $y' + y^2 = 0$ avec la condition initiale $y(0) = 1$. La solution analytique s'écrit $y = 1/(t + 1)$ et sert de référence pour tester la méthode d'Euler. La récurrence (10) s'écrit dans ce cas particulier :

$$Y_{n+1} = Y_n - k Y_n^2 \quad (11)$$

i	t_i	y_n exact	y_n num	Erreur (%)
1	0.1	0.909	0.9	1.0
2	0.2	0.833	0.819	1.7
3	0.3	0.769	0.752	2.3
4	0.4	0.714	0.695	2.7
5	0.5	0.666	0.647	3.0

TABLE 1 – comparaison avec la solution analytique

Les premières valeurs de Y_n analytique et numérique ainsi que l'erreur globale, sont résumées dans le tableau 1. Retrouvez-les à titre d'exercice.

D'après la figure 2, on remarque que l'erreur globale en fixant k constant, augmente avec le nombre d'itérations. La seule solution pour obtenir des valeurs plus précises avec cette méthode est de réduire le pas de temps k , quitte à augmenter le nombre total d'itérations pour atteindre le temps final T .

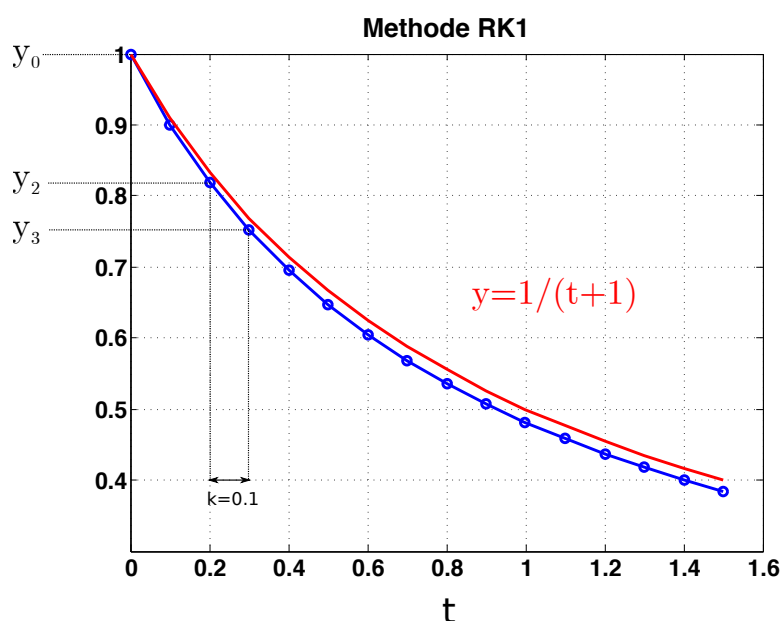


FIGURE 2 – Intégration par la méthode d'Euler (RK1)

3.2 Implémentation de la méthode d'Euler en Matlab

On propose l'implémentation de l'intégrateur $RK1$ suivante en langage Matlab. La variable f est le vecteur second membre du système d'équations différentielles qui est défini dans une fonction Matlab annexe.

Listing 1 – integrateur rk1

```
function [tvec, yvec]=rk1(f, param, y0, options)
```

```
% rk1(@equadif,[t0 tf k],[y0(1) y0(2) ... y0(n)], options);
% methode d'Euler explicite ordre 1
%
5
% intervalle d'etude en temps et pas
t0 = param(1);
tf = param(2);
k = param(3);
10
% sauvegarde du vecteur y0
% dans la 1ere ligne du tableau yvec
n=1;
tvec(n) = t0;
15 yvec(n, :) = y0;

t = t0;
y = y0;

20 while (t<tf)
    y=y+k*f(t, y);

    % pas suivant
    n=n+1;
25 t=t+k;

    % stockage dans des tableaux dynamiques
    tvec(n) = t;
    yvec(n, :)= y;
30
end
```

On propose maintenant d'intégrer les équations de la dynamique d'un oscillateur harmonique amorti. On le lâche à partir d'une position $x_0 = 1m$ fixée, sans vitesse initiale $v_0 = 0m/s$.

Listing 2 – exemple d'appel de rk1

```
% script solver_rk1

clc % efface la fenetre de commande
clear all % efface la memoire
5 close all % ferme les fenetres graphiques

global omega0
global eta

10 omega0=1; eta=0.5;
```

```

param=[0 25 0.05];
[T,Y] = rk1(@oscil,param,[1 0]);

15 x=Y(:,1); % recuperation de la position
    v=Y(:,2); % et de la vitesse

    plot(T, x, 'r+-'); hold on;
    plot(T, v, 'bo-')
20 hold on

    title(['Methode RK1 k=' num2str(param(3))])

```

Listing 3 – implémentation du système d'équations différentielles associées à l'oscillateur harmonique

```

function dY_dt = oscil(t,Y)
dY_dt = zeros(1,2); % vecteur ligne

global omega0 % variables globales
5 global eta

    % changement de variable
    % pour garder les notations de l'equation
    x=Y(1);
10 v=Y(2);

    % ecriture de l'equation telle quelle
    dx_dt = v;
    dv_dt = -(omega0^2)*x-eta*v;

15 % changement de variable pour
    % se conformer au vecteur de sortie dY_dt
    dY_dt(1)=dx_dt;
    dY_dt(2)=dv_dt;

20 end

```

3.3 Avantages et inconvénients de la méthode d'Euler

La relation de récurrence (10) est facile à implémenter et elle permet de traiter un système d'équations différentielles linéaires voire non-linéaires, écrites de manière explicite. Le coût en temps de calcul est faible car seule une seule évaluation de $f(t, y)$ est faible à chaque pas.

Mais, comme le schéma est d'ordre 1, sa précision reste faible. L'erreur de troncature à chaque pas est en $\frac{1}{2}k^2 \frac{d^2 Y}{dt^2}$. Cela revient pour un système mécanique à oublier le terme d'inertie à chaque pas !

De plus, il faut être prudent dans l'utilisation de ce schéma, car il n'est stable que pour des pas de temps en dessous d'une valeur critique k_c . Notez de plus que k_c ne pourra être déterminé théoriquement que pour des systèmes d'équations différentielles linéaires. En pratique, pour un système non-linéaire, il sera nécessaire d'étudier la convergence d'une observable comme l'énergie ou des composantes du vecteur Y à un instant fixé, en réalisant des simulations avec des pas de temps diminuant de manière dyadique ($k_n \propto 1/2^n$).

3.4 Etude de la stabilité du schéma d'Euler

L'étude de stabilité n'est réalisable que si la fonction $f(t, Y) = AY$ suit une loi linéaire voire affine. On va établir ici la condition suffisante sur le pas de temps pour que le bruit sur l'inconnue soit atténué au cours de l'intégration.

Imaginons que l'on ait à notre disposition un ordinateur permettant de faire du calcul exact. Il serait capable de calculer la récurrence $Y_{n+1} = Y_n + kAY_n$ sans introduire d'erreur.

Malheureusement, dans une machine réelle, les nombres sont approximés par un nombre fini de bits. Une erreur de troncature ϵ_n à l'instant n se propagera à l'instant $n + 1$ selon :

$$Y_{n+1} + \epsilon_{n+1} = Y_n + \epsilon_n + kA(Y_n + \epsilon_n) \quad (12)$$

En soustrayant membre à membre, la relation de récurrence, on en déduit une relation de récurrence pour l'erreur :

$$\epsilon_{n+1} = (1 + kA)\epsilon_n \quad (13)$$

ou encore, en fonction d'un bruit initial ϵ_0 , — dès l'affectation des conditions initiales —, l'erreur de troncature à l'instant n suit une loi de puissance (matricielle) :

$$\epsilon_n = (1 + kA)^n \epsilon_0 = G^n \epsilon_0 \quad (14)$$

où G est une matrice de gain.

En supposant G diagonalisable, on peut l'exprimer en fonction de ses valeurs propres :

$$G^n = P(I + k\Lambda)^n P^{-1} \quad (15)$$

Le schéma d'Euler est stable s'il y a atténuation à chaque pas de temps, autrement dit, il suffit que $|1 + k\lambda_i| \leq 1$ pour toute valeur propre de la matrice A .

En pratique, on peut rencontrer les cas suivants :

1. s'il existe au moins une valeur propre $\lambda_i > 0$ alors le schéma est instable,
2. si toutes les valeurs propres sont réelles et strictement négatives alors le pas de temps doit être choisi selon le critère : $k < 2/\max_i |\lambda_i|$,
3. si les valeurs propres sont complexes avec une partie réelle strictement négative alors il faut prendre $k < \min \frac{-2\Re(\lambda_i)}{|\lambda_i|^2}$ pour que les gains soient petits par rapport à 1.

4 Tangente améliorée (Runge-Kutta d'ordre 2)

La première idée pour pallier le manque de précision de la méthode d'Euler (RK1), est de prendre en compte un terme supplémentaire dans le développement limité en temps :

$$Y_{n+1} = Y_n + k \left. \frac{dY}{dt} \right|_n + \frac{k^2}{2} \left. \frac{d^2Y}{dt^2} \right|_n + \vartheta(k^3) \quad (16)$$

et d'exprimer les dérivées en fonction de $f(t, Y)$:

$$Y_{n+1} = Y_n + k f(t_n, Y_n) + \frac{k^2}{2} \frac{df}{dt}(t_n, Y_n) + \vartheta(k^3) \quad (17)$$

Cette approche est fonctionnelle à condition que $\frac{df}{dt}$ soit facilement calculable. comme en mécanique du point, où cette dérivée correspond à l'accélération. Mais, de manière plus générale, cette méthode vient vite compliquée à mettre en oeuvre.

La seconde idée est de combiner des estimations de $f(t, Y)$ en deux instants différents dans l'intervalle $[t_n, t_n + k]$ pour écrire un schéma d'ordre 2 selon :

$$Y_{n+1} = Y_n + a K_1 + b K_2 + \vartheta(k^3) \text{ avec } \begin{cases} K_1 &= k f(t_n, Y_n) \\ K_2 &= k f(t_n + \alpha k, Y_n + \beta K_1) \end{cases} \quad (18)$$

Comme (α, β) sont petits par rapport à 1, un développement de Taylor à l'ordre 2 de K_2 donne :

$$f(t_n + \alpha k, Y_n + \beta K_1) = f(t_n, Y_n) + \alpha k \partial_t f(t_n, Y_n) + \beta K_1 \partial_y f(t_n, Y_n) + \vartheta(k^2) \quad (19)$$

Après substitution complète dans (18), on obtient le développement suivant :

$$Y_{n+1} = Y_n + (a + b) k f(t_n, Y_n) + k^2 (b \alpha \partial_t f(t_n, Y_n) + b \beta f(t_n, Y_n) \partial_y f(t_n, Y_n)) + \vartheta(k^3) \quad (20)$$

qui doit coïncider avec le développement de Taylor à l'ordre 2 de (17) :

$$Y_{n+1} = Y_n + k f(t_n, Y_n) + \frac{k^2}{2} (\partial_t f(t_n, Y_n) + f(t_n, Y_n) \partial_y f(t_n, Y_n)) + \vartheta(k^3) \quad (21)$$

En comparant (20) et (21), on obtient le système d'équations suivant :

$$\begin{cases} a + b &= 1 \\ b \alpha &= 1/2 \\ b \beta &= 1/2 \end{cases} \quad (22)$$

Il est sous-déterminé car on n'a seulement que 3 équations pour 4 inconnues. Pour rendre le système kramérien et pour limiter le nombre d'opérations arithmétiques, on fixe $a = 0$. Sa résolution donne $b = 1$, $\beta = 1/2$ et $\alpha = 1/2$.

Finalement, le schéma d'intégration Runge-Kutta d'ordre 2 s'écrit :

$$Y_{n+1} = Y_n + k f(t_n + \frac{k}{2}, Y_n + \frac{k}{2} f(t_n, Y_n)) \quad (23)$$

qu'on implémentera plutôt en introduisant deux variables intermédiaires t_{mid} et Y_{mid}

$$\begin{cases} t_{mid} &= t_n + \frac{k}{2} \\ Y_{mid} &= Y_n + \frac{k}{2} f(t_n, Y_n) \\ Y_{n+1} &= Y_n + k f(t_{mid}, Y_{mid}) \end{cases} \quad (24)$$

4.1 Interprétation graphique du schéma RK2

Le schéma RK2 implémente déjà une méthode prédictor-correcteur explicite :

1. On estime d'abord la valeur de Y_{mid} en appliquant un schéma d'Euler avec un pas $k/2$.
2. On corrige la pente au point milieu en utilisant l'estimation précédente.
3. On calcule finalement l'accroissement de Y entre les instants t_n et t_{n+1} en utilisant la valeur de la pente corrigée au point milieu.

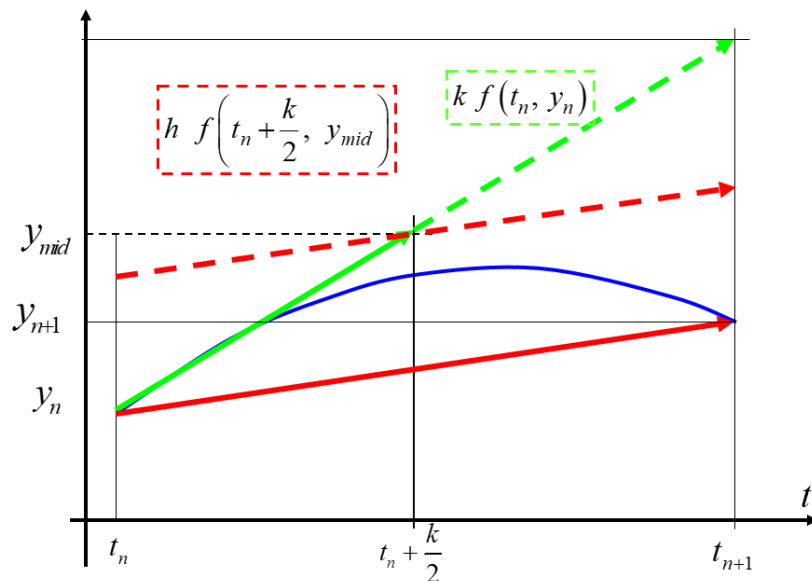


FIGURE 3 – Intégration par la méthode RK2

4.2 Etude de la stabilité de RK2

On rappelle que l'étude de stabilité n'est réalisable que si la fonction $f(t, Y) = AY$ suit une loi linéaire voire affine. Dans ce cas, intégrer l'équation différentielle revient à appliquer la relation de récurrence :

$$Y_{n+1} = Y_n + kAY_n + \frac{(kA)^2}{2} Y_n. \quad (25)$$

Dans un ordinateur, les nombres sont approximés par un nombre fini de bits. Une erreur de troncature ϵ_n à l'instant n se propagera à l'instant $n + 1$ selon :

$$Y_{n+1} + \epsilon_{n+1} = Y_n + \epsilon_n + kA (Y_n + \epsilon_n) + \frac{(kA)^2}{2}(Y_n + \epsilon_n) \quad (26)$$

En soustrayant membre à membre, la relation de récurrence, on en déduit une relation de récurrence pour l'erreur :

$$\epsilon_{n+1} = \left(1 + kA + \frac{(kA)^2}{2}\right) \epsilon_n \quad (27)$$

ou encore, en fonction d'un bruit initial ϵ_0 , — dès l'affectation des conditions initiales —, l'erreur de troncature à l'instant n suit une loi de puissance (matricielle) :

$$\epsilon_n = \left(1 + kA + \frac{(kA)^2}{2}\right)^n \epsilon_0 = G^n \epsilon_0 \quad (28)$$

où G est une matrice de gain.

En supposant G diagonalisable, on peut l'exprimer en fonction de ses valeurs propres :

$$G^n = P \left(I + k\Lambda + \frac{(k\Lambda)^2}{2} \right)^n P^{-1} \quad (29)$$

Le schéma RK2 est stable s'il y a atténuation à chaque pas de temps, autrement dit, il suffit que $|1 + k\lambda_i + \frac{(k\lambda_i)^2}{2}| \leq 1$ pour toute valeur propre de la matrice A .

En pratique, on peut rencontrer les cas suivants :

1. s'il existe au moins une valeur propre $\lambda_i > 0$ alors le schéma est instable,
2. si toutes les valeurs propres sont réelles et strictement négatives alors le pas de temps doit être choisi selon le critère : $k < 2/\max_i |\lambda_i|$,
3. si les valeurs propres sont complexes avec une partie réelle strictement négative alors une étude graphique permet de choisir un pas de temps pour lequel tous les gains ont un module plus petit que 1.

5 Intégrateurs à pas adaptatif

On présente ici une famille de méthodes d'intégration des équations différentielles plus versatile gérant elle-même le choix du pas de temps d'intégration k .

L'utilisateur doit d'abord bien connaître la physique du problème qu'il traite car il doit définir une valeur acceptable ϵ_m de l'erreur d'intégration à chaque pas.

A chaque itération, l'idée principale est d'ajuster le pas d'intégration de manière à ce que l'erreur ϵ ne dépasse pas la valeur de l'erreur acceptable ϵ_m . L'algorithme consiste à effectuer l'intégration du système d'équations différentielles de l'instant t_n à l'instant $t_n + k$, deux fois avec deux méthodes différentes, la seconde étant d'un ordre de précision supérieur à la première.

Chacune donne une estimation de $Y(t_n + k)$ que l'on note A_1 et A_2 qui nous serviront à calculer une estimation de l'erreur commise lors de l'intégration.

5.1 Méthode d'Euler-Euler améliorée RK12

Pour notre démonstration, on suppose connaître la fonction $\Phi(t)$, solution analytique de notre système d'équations différentielles $\frac{dY}{dt} = f(t, Y)$.

Sans perdre en généralité, on suppose qu'à l'instant n , les solutions analytique et numérique coïncident : $\phi(t_n) \equiv Y_n$. Dans la phase 1 de l'algorithme, on applique la méthode d'Euler d'ordre 1 avec un pas d'intégration h . On obtient :

$$A_1 = Y_n + k f(t_n, Y_n) \quad (30)$$

or, après développement et substitution au voisinage de t_n , on a :

$$\begin{aligned} \phi(t_n + h) &= \phi(t_n) + k \phi'(t_n) + \frac{k^2}{2} \phi''(t_n) + \vartheta(k^3) \\ &= Y_n + k f(t_n, Y_n) + \frac{k^2}{2} \phi''(t_n) + \vartheta(k^3) \end{aligned} \quad (31)$$

Ceci implique que

$$A_1 = \phi(t_n + k) - \frac{k^2}{2} \phi''(t_n) + \vartheta(k^3) \quad (32)$$

Dans la phase 2, on applique la méthode d'Euler améliorée (autrement dit RK2), on sait que :

$$A_2 = \phi(t_n + k) + \vartheta(k^3) \quad (33)$$

Par différence des estimations A_1 et A_2 , on déduit l'erreur d'intégration :

$$\epsilon = |A_1 - A_2| = \frac{k^2}{2} |\phi''(t_n)| \quad (34)$$

On obtient l'algorithme de principe suivant :

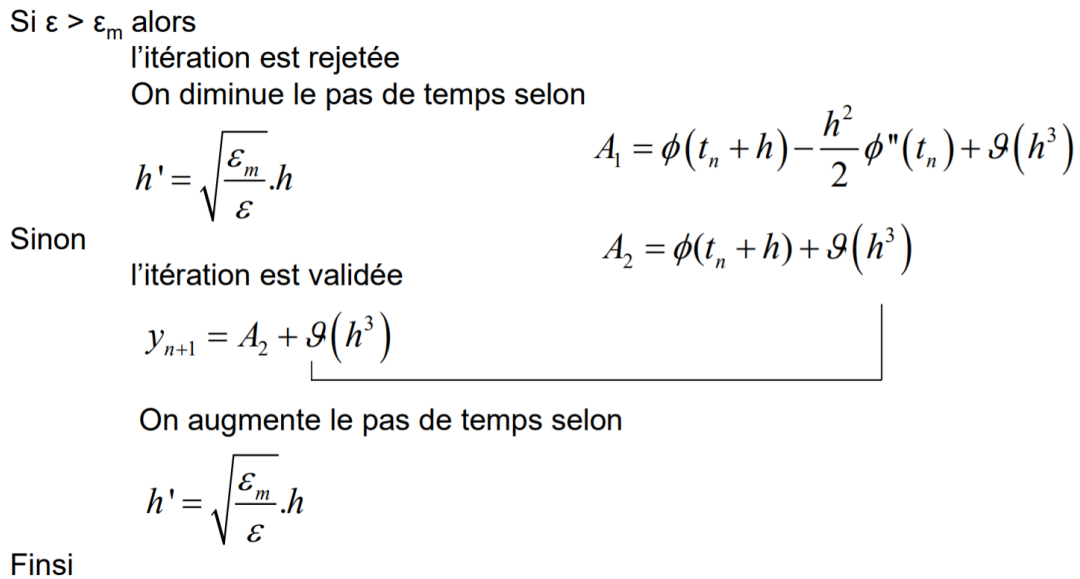


FIGURE 4 – Algorithme de la méthode RK12

Une implémentation de l'algorithme en Matlab vous est présentée ci-dessous :

Listing 4 – Implémentation de l'algorithme

```

% rk_adapt12(@equadif,[ti tf dt],[y1i y2i y3i], options);
% methode d'Euler a pas adaptatif
%
5 function [tvec, yvec]=rk_adapt12(f, param, y0, options)
AbsTol=1e-6;
% intervalle d'etude en temps et pas
t0 = param(1);
tf = param(2);
10 h = param(3);

% sauvegarde du vecteur y0 dans la 1ere ligne du tableau yvec
n=1;
tvec(n) = t0;
15 yvec(n, :) = y0;

t = t0;
y = y0;

20 while (t<tf)
    f1=f(t, y);
    f2=f(t+h/2., y+h/2.*k1);

```

```
25     A1=y + h*f1;      % RK1
    A2=y + h*f2;      % RK2

    E=A1-A2;
    norm_E=norm(E, inf);
    h=0.9*sqrt(AbsTol/norm_E)*h;

30     if (norm_E <= AbsTol)
        % pas suivant
        t=t+h;
        n=n+1;
35         y=A2;

    % stockage
        tvec(n)=t;
        yvec(n, :)=y;
40     end
end
```

En pratique, on utilisera des méthodes plus précises comme RK45 proposée par Fehlberg en 1969 qui est une méthode d'ordre $\vartheta(k^4)$ avec un estimateur d'erreur d'ordre $\vartheta(k^5)$. Une implémentation sous Matlab s'appelle ODE45.